

Classifiers KNN – SVM – MLP (and CNN)

José Miguel Luís Antunes | no. 202321128 | PCIA – Project 2
Code: https://github.com/migasantunes/Classifiers_KNN_SVM_MLP

1 Goals

This project's goal is to construct, train and analyze performances between the following classifiers: K Nearest Neighbors (KNN), Support Vector Machines (SVM) and Multi-Layer Perceptron (MLP). The goal of the extra is to do the same with the MNIST dataset and a Convolutional Neural Network (CNN) classifier.

2 Data and Methodology

Iris dataset. As a first step, the KNN classifier was tested on the Iris dataset (80/20 stratified split), with an accuracy of 1.0.

Digits dataset. 1797 images of 8×8 pixels, flattened into 64 features with values in the 0–16 range. The 10 classes are balanced (174 to 183 samples each). Figure 1 shows the first 10 images with their labels.

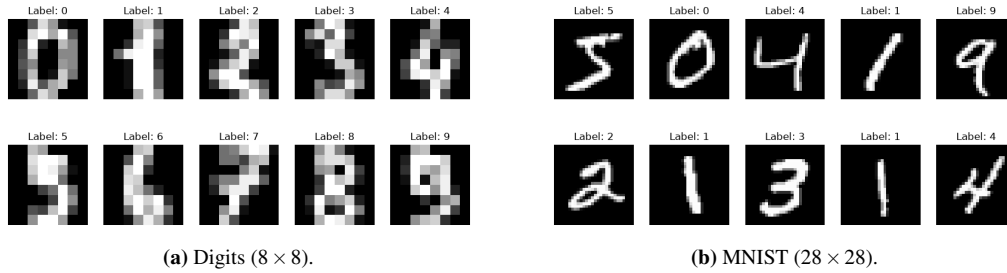


Figure 1: First 10 images of each dataset with their labels.

Protocol. The data was split into 80% training and 20% test (360 images), stratified, with a fixed seed. Each classifier was placed in a Pipeline, tuned with GridSearchCV (5-fold CV) on the training set and the best model was then evaluated on the test set. Since the digits dataset has a small range of values, using StandardScaler to scale the data may not be necessary. Either way, both options were included in the grid to see if there are any performance differences. Also, since GridSearchCV is used, there is no need to implement stratification, because it is done automatically. The grids were:

- **KNN:** $n_neighbors \in \{1, 3, 5, 7\}$, $weights \in \{uniform, distance\}$.
- **SVM:** RBF kernel with $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{scale, 0.01, 0.001, 0.0005, 0.0001\}$; linear kernel with the same C values. At first C 's highest value was 10 and γ 's lowest was 0.001 and the grid search chose those 2 values, so 100 and $\{0.0005, 0.0001\}$ were added to see if it was because of the boundaries. The best parameters became $C = 1$ and $\gamma = 0.0005$, which means they moved inside the grid and the boundaries were affecting the results.
- **MLP:** $hidden_layer_sizes \in \{(64), (128), (128, 64)\}$, $\alpha \in \{10^{-4}, 10^{-3}, 10^{-2}\}$, $max_iter = 1000$, fixed seed.

For each model the CV mean and standard deviation, test accuracy, macro F1, classification report, confusion matrix, training time and prediction time per sample were gathered.

3 Results – Digits Dataset

Table 1: Best model of each classifier on the digits dataset.

	Best parameters	Scaled	CV mean	CV std	Test acc.	F1	Train (ms)/ Pred. per sample (ms)
KNN	$n_neighbors = 1$, uniform	No	0.9902	0.0081	0.9861	0.9859	1.15 / 0.0381
SVM	$C = 1$, $\gamma = 0.0005$, RBF	No	0.9903	0.0060	0.9917	0.9916	31.24 / 0.0579
MLP	$\alpha = 10^{-4}$, (64)	Yes	0.9784	0.0095	0.9806	0.9803	896.06 / 0.0028

Table 2: Best CV score with and without StandardScaler.

	Best CV Scaled	Best CV Unscaled	Winner	Difference
KNN	0.9749	0.9902	Unscaled	0.0153
SVM	0.9833	0.9903	Unscaled	0.0070
MLP	0.9784	0.9729	Scaled	0.0056

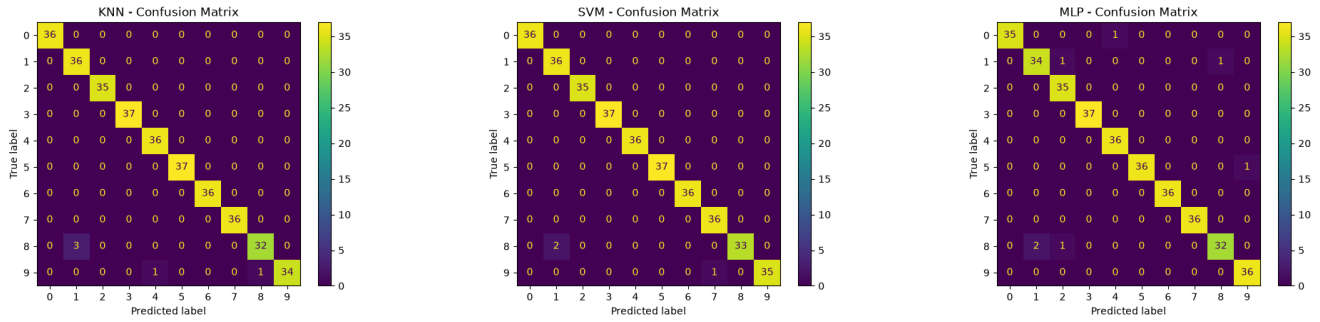


Figure 2: Confusion matrices on the digits test set.

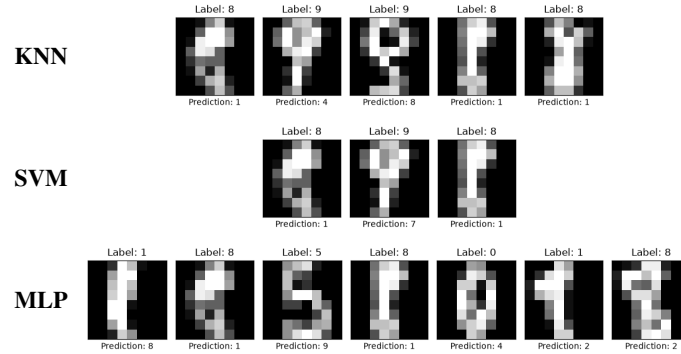


Figure 3: All misclassified digits of each classifier (true label on top, prediction below).

Accuracy: All three classifiers perform very well on the digits dataset (98–99%). On the test set, SVM was best (99.2%, 3 errors), followed by KNN (98.6%, 5 errors) and MLP (98.1%, 7 errors). For the cross-validation, SVM and KNN are practically tied (0.9903 vs 0.9902), while MLP is about 1.2 points lower (0.978). With only 360 test images, the 2-image gap between SVM and KNN is not conclusive on its own, so the CV scores are the stronger evidence.

F1 vs accuracy: F1 is almost identical to accuracy for every model, which is expected because the classes are balanced, making accuracy a fair metric.

Stability: SVM has the lowest CV standard deviation (0.0060), meaning its performance depends the least on which data it is trained on. MLP is the least stable (0.0095).

Best hyperparameters:

- KNN chose $k = 1$: only the nearest training image is enough, which probably indicates each digit class forms tight and clean clusters. With a single neighbour the choice of weights is irrelevant, since there is nothing to weight, so “uniform” and “distance” give identical predictions and the same CV score. `GridSearchCV` simply keeps the first option in the grid.
- SVM chose the RBF kernel over the linear one, so non-linear decision boundaries help. The very small γ of 0.0005 makes sense for unscaled pixels, since squared distances between images are large. It also chose $C = 1$: it underfits with $C = 0.1$ and doesn’t improve with $C = 10$ or 100 .
- MLP chose the smallest network, with one hidden layer of 64 neurons, and the smallest regularization: it’s a small dataset, so there is no need for a large network, and stronger regularization only reduced accuracy.

Scaling: For this dataset, using `StandardScaler` has a negative effect on KNN and SVM and slightly helps MLP (Table 2). This is because all the pixels are in the same 0–16 range and with scaling, it makes every pixel’s variance equal. The border pixels are almost always 0, so their standard deviation is tiny and scaling blows up their rare non-zero values. Since KNN and SVM are distance-based classifiers, they don’t have weight mechanisms like MLP to give more importance to the pixels that are more relevant to the classification.

Computational cost: The models show opposite trade-offs:

- Training: KNN ≈ 1 ms, SVM ≈ 31 ms, MLP ≈ 900 ms. MLP is almost $800\times$ slower than KNN.
- Prediction (whole test set): KNN ≈ 14 ms, SVM ≈ 21 ms, MLP ≈ 1 ms. MLP is instant compared to KNN and SVM.

KNN learns quickly but predicts slowly and MLP trains slowly but predicts quickly. SVM is in the middle for both.

Learning curves: The MLP training accuracy reaches 1.0 around epoch 50 and the validation accuracy stabilizes at about 0.97–0.98, while the validation loss keeps slowly decreasing and never rises (Figure 5a). There is a small gap but no overfitting, and the training with `GridSearchCV` stopped at epoch 165 of the 1000 allowed.

Mislabeled images: Some of the mislabeled images are simply hard to classify even for humans, maybe also due to the resolution (Figure 3). Two images were mislabeled by all 3 classifiers, both 8’s misclassified as 1’s.

4 Extra – MNIST Dataset and CNN

Setup. MNIST has 70 000 images of 28×28 pixels (784 features), with classes roughly balanced (6313 to 7877 samples each). The data was split into 80% training and 20% test (14 000 images), and 20% of the training set was kept for validation (44 800 training and 11 200 validation images). The train/validation/test split isn't needed for KNN and SVM, but since it is used in the MLP and CNN curves to understand what epochs to use, it was used for all classifiers, to compare the results in identical conditions. GridSearchCV was not used, since the goal is just to show the performance of these classifiers with a large dataset. The hyperparameters were set by what I believe is the best for each classifier, based on the previous runs, but nothing too fine-tuned. The data was scaled for MLP only, as explained by the run on the digits dataset.

CNN. Two convolutional blocks (Conv 3×3 with 32 and 64 filters, ReLU, MaxPool 2×2), followed by Dropout, a dense layer of 128 neurons, Dropout and the output layer. It was trained in PyTorch on the GPU (CUDA) with Adam (lr = 0.001), batch size 128 and 20 epochs. It was not tested without scaling, since the CNN is a neural network and it will definitely benefit from scaling.

Table 3: Results on MNIST (11 200 validation and 14 000 test images).

	Parameters	Val. acc.	Test acc.	F1	Training time (s)	Pred. per sample (ms)
KNN	n_neighbors = 3, distance	0.9719	0.9706	0.9705	0.0023	0.8073
SVM	$C = 10$, $\gamma = \text{scale}$, RBF	0.9802	0.9824	0.9822	111.79	5.3350
MLP	(128, 64), $\alpha = 0.1$, lr = 10^{-4} , scaled	0.9759	0.9760	0.9759	269.29	0.0065
CNN	2 conv blocks + dense (128)	0.9903	0.9914	0.9915	31.00 (GPU)	0.0123 (GPU)

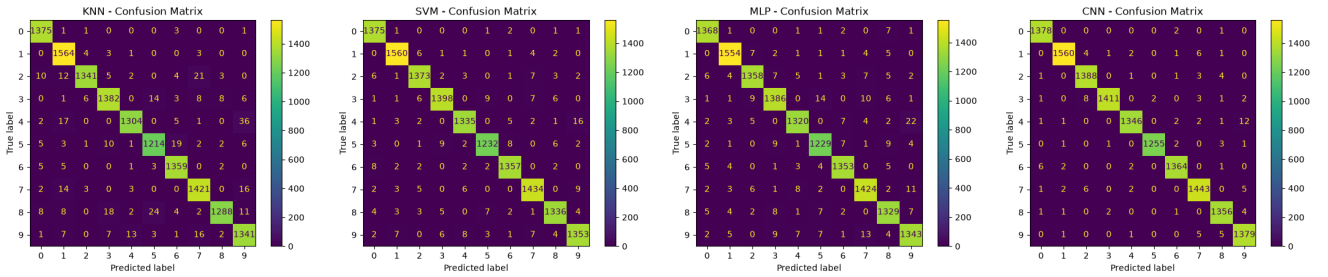


Figure 4: Confusion matrices on the MNIST test set.

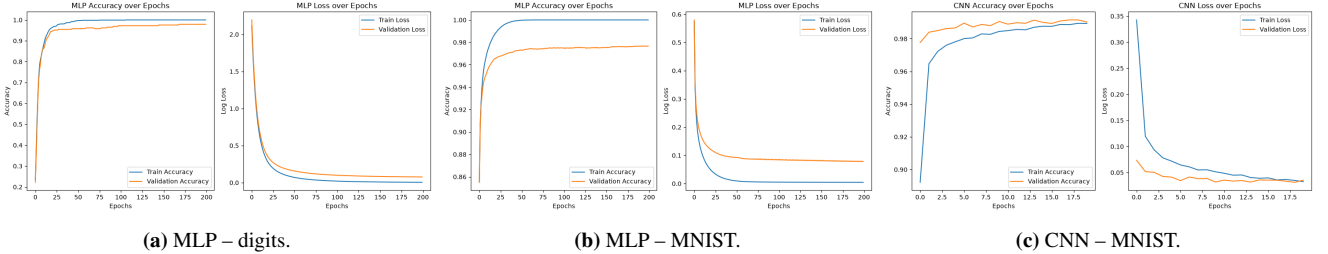


Figure 5: Training and validation accuracy and loss over epochs.

Accuracy: CNN (99.14%) > SVM (98.24%) > MLP (97.60%) > KNN (97.06%). CNN is the only model that uses the 2D structure of the image, which makes it detect local patterns anywhere in the image, while KNN, SVM and MLP see 784 independent numbers.

Validation vs test: The accuracies differ by about 0.2 percentage points at most for every model, so the validation set was a reliable estimate and no model overfitted to it.

Compared with the digits dataset: The SVM is still the best of the classical models, but the ranking of the other two flipped. KNN was second on digits and is now the worst. MLP becomes better the more data it has, thanks to pattern recognition.

Training time: KNN only stores the data, making it instant, while SVM (112 s) and MLP (269 s) require substantial computation. The CNN trained in 31 s with the GPU.

Prediction time: KNN compares each image with all the training images (0.81 ms), and the SVM evaluates the kernel against thousands of support vectors (5.3 ms, $\sim 430\times$ slower than the CNN). The MLP (0.0065 ms, CPU) and the CNN (0.012 ms, GPU) are just a few matrix multiplications, so they are practically instant.

Why KNN is too slow: KNN is a lazy learner: all the work is left for prediction. Each test image is compared with all 44 800 training images of 784 pixels, and the whole training set must be kept in memory. Its prediction time per sample grew about $21\times$ from digits to MNIST (0.038 to 0.81 ms) and keeps growing with the size of the training set, while the prediction cost of MLP and CNN does not depend on it. The same is true for the SVM through its support vectors, which is why it is even slower here.

MLP curves: The MNIST MLP reaches a training accuracy of 1.0 around epoch 40, while validation accuracy stabilizes at about 0.975 and validation loss stays flat around 0.08 (Figure 5b), which agrees with the training stopping at epoch 151.

CNN curves: In the early epochs, validation accuracy is higher than training accuracy, and the same with loss but in the opposite direction (Figure 5c). That's because training metrics are measured while Dropout is switched on, and averaged over the whole epoch, including the start when the model was still worse, while validation is measured at the end of the epoch, with Dropout off. The accuracy and loss curves are as expected, converging around 20 epochs (checked the graph with 100 epochs and 20 is enough) and not overfitting.

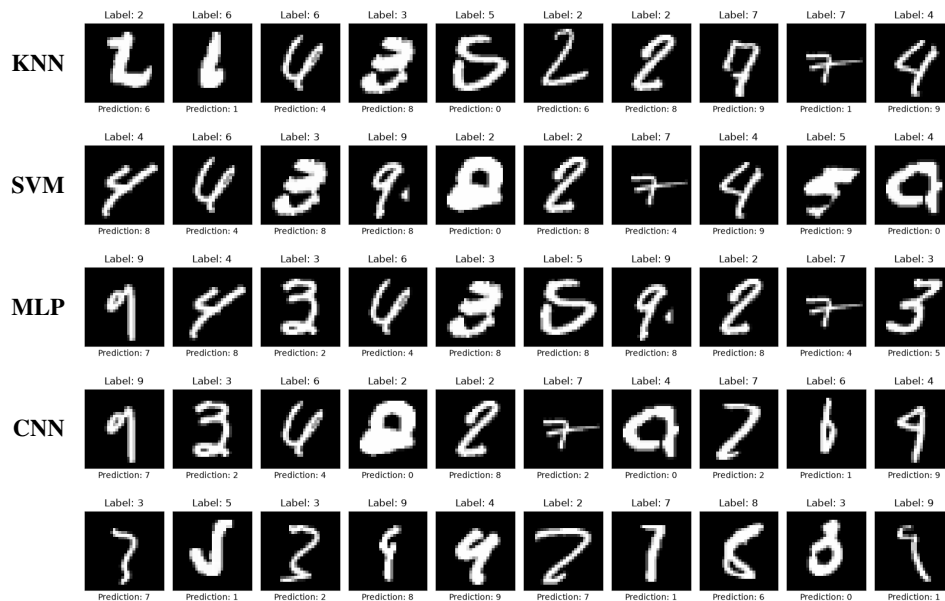


Figure 6: First misclassified MNIST test images of each classifier (true label on top, prediction below). Total errors out of 14 000: KNN 411, SVM 247, MLP 336, CNN 120.

Mislabeled images: Most of the CNN's errors are between digits with similar shapes, like 4 – 9 or 7 – 2, and many of them are badly written and hard to classify even for a human (Figure 6).

5 Conclusions

On the digits dataset, SVM gives the best balance: the highest test accuracy, the most stable CV scores and a reasonable cost. MLP would be preferred if prediction speed mattered most, since it predicts a lot faster for about 1 point less accuracy. KNN is the simplest strong baseline, but it scales the worst to larger datasets, as MNIST confirmed. For image data, the CNN is the clear choice, being the most accurate and fast at prediction. Among the classical models, the SVM gives the best accuracy but is the slowest to use, and the MLP is the best balance once trained.

Use of AI: AI was used for planning, best values to start with for the parameters, helping with understanding some concepts, a classifier code jumpstart (just a simple code snippet), a code quality review that resulted in changes with the AI's feedback, and the training/validation loss and accuracy plotting code. Everything else was done entirely by me, including code, with the use of the *sklearn*, *pytorch* and other documentation and research.